SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5116

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage**:** 10%

QUESTIONS: **Total Marks: 50**

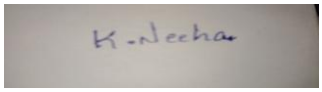
Annexure A

Debugging & Optimization Task

Code of Conduct:

I K.Neeha (192425245) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)
A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures.
B. Refactor the code to minimize redundant computations and memory usage.
C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation
A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.
B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.
B. Optimize exponentiation operations for large prime values.
C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic.
B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understandin g	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrate s strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrate d
Code Quality	15%	Produces clean, well-structured code with proper comments and documentatio n	Code is mostly clear with minor documentatio n gaps	Code lacks structure and proper documentatio n	Poorly written code with no documentati on

1. Introduction

Block cipher modes of operation define how a block cipher, such as AES (Advanced Encryption Standard), encrypts data longer than a single block. Different modes provide different levels of performance, memory usage, and error propagation.

ECB, CBC, CFB, and OFB are four traditional modes of operation. An inefficient implementation may experience high latency, unnecessary memory allocation, incorrect padding, or decryption failures due to improper Initialization Vector (IV) handling.

The objective is to debug these issues, optimize the Python implementation, and identify a suitable mode for real-time secure communication.

2. Objectives

1. Identify and correct padding and IV-handling errors.
2. Reduce redundant computations and unnecessary memory usage.
3. Implement and compare ECB, CBC, CFB, and OFB.
4. Measure encryption and decryption time.
5. Recommend an appropriate mode for real-time communication**.**

3. Problems and Debugging Methods

Problem	Cause	Solution
Decryption failure	Incorrect padding or unpadding	Apply padding and remove it only for ECB and CBC
IV mismatch	Different IV used during decryption	Store and reuse the same IV for decryption
Invalid block size	ECB/CBC input is not block-aligned	Use PKCS#7 padding
High latency	Repeated cipher creation and redundant operations	Avoid unnecessary conversions and repeated processing
High memory usage	Creating unnecessary intermediate copies	Process data in chunks for large messages
Incorrect output	Encryption and decryption configurations differ	Use matching keys, modes, and IVs

Important: AES has a block size of 16 bytes. ECB and CBC require complete blocks, so padding is generally necessary for arbitrary-length input. CFB and OFB can process arbitrary-length data without block padding**.**

4. Optimized Python Implementation

The following implementation uses the pycryptodome library. It measures encryption and decryption time, verifies the decrypted result, and reports ciphertext size.

Download the complete Python program:

Download block_cipher_modes_analysis.py

Run it in Google Colab or Python after installing the dependency:

!pip install pycryptodome

The program uses a single randomly generated AES key for the benchmark, generates a fresh IV for each non-ECB encryption, and reuses that IV for the corresponding decryption.

5. Flowchart of the Implementation

Start

Input plaintext and AES key

Select mode: ECB, CBC, CFB or OFB

Generate IV where required and apply padding if needed

Encrypt and record execution time

Decrypt using the same key and IV

Verify plaintext and compare performance

6. Comparison of Block Cipher Modes

Feature	ECB	CBC	CFB	OFB
IV required	No	Yes	Yes	Yes
Padding required	Yes	Yes	No	No
Blocks encrypted independently	Yes	No	No	No
Encryption parallelism	Yes	No	Limited	Keystream generation can be sequential
Error propagation	Limited to affected block	Can affect adjacent plaintext blocks	May affect multiple segments	Usually affects corresponding bit positions
Pattern security	Poor	Better than ECB when used correctly	Better than ECB when used correctly	Better than ECB when used correctly
Suitable for new applications	No	Limited use	Limited use	Limited use

ECB reveals patterns in repeated plaintext blocks and should not be used for sensitive messages. CBC, CFB, and OFB also do not provide authentication on their own.

7. Performance Analysis

The implementation measures encryption and decryption time using Python's perf_counter() function. The actual results depend on the processor, library implementation, message size, and operating environment.

Performance metric	Meaning
Encryption time	Time taken to encrypt the input
Decryption time	Time taken to recover the plaintext
Ciphertext size	Number of bytes produced after encryption
Correctness	Whether decrypted data matches the original input
Memory consumption	Memory required during encryption and decryption

For a fair comparison, run multiple trials with identical data sizes, calculate average times, and measure memory usage separately. Avoid declaring a universal fastest mode without benchmark

results.

8. Which Mode Is Best for Real-Time Communication?

For a new real-time secure communication system, AES-GCM is generally recommended rather than choosing ECB, CBC, CFB, or OFB. AES-GCM provides both confidentiality and integrity, and it can be highly efficient on supported hardware.

Requirement	Recommendation
Low latency	AES-GCM with hardware acceleration where available
Confidentiality and integrity	AES-GCM
Large data processing	Process data in chunks with appropriate authentication
Legacy compatibility	CBC with a separate, correctly designed authentication mechanism
Avoiding pattern leakage	Do not use ECB
Memory efficiency	Avoid unnecessary data copies and use bounded buffers

For secure use of AES-GCM, never reuse a nonce with the same key. For chunked communication, each chunk needs a correctly managed nonce and authentication strategy.

9. Conclusion

The main causes of decryption failure in block cipher implementations are incorrect padding, mismatched IVs, and inconsistent encryption/decryption configurations. Correct padding rules, proper IV handling, and reduced intermediate memory allocations improve reliability and performance.

ECB, CBC, CFB, and OFB have different operational characteristics, so benchmarking is necessary to compare their actual performance. However, for modern real-time secure communication, AES-GCM is generally the preferred option because it combines efficient encryption with authentication and integrity protection.